Programming Principles I

Week 5 · Class 9

Linear arrays

Many values under one name · index 0 to N-1 · a loop over every box

```
int scores[5] = {7, 9, 6, 10, 8};
for (int i = 0; i < 5; i++) {
    printf("%d ", scores[i]);
}
```

Quiz 2 is back

Three ideas cost most of the marks in this room.

Tracing a loop. The counting program. The small things.

All three come back today, inside every array loop.

Your mark and your marked paper are in Blackboard. Read my comments at home, not now.

Mistake 1: trace the loop, round by round

```c
int x = 1;
int count = 0;
while (x < 20) {
    printf("%d ", x);
    if (x % 3 == 0) {
        count++;
    }
    x = x * 2 + 1;
}
printf("\nx = %d, count = %d\n", x, count);
```

x	x < 20?	prints	count
1	yes	1	0
3	yes	3	1
7	yes	7	1
15	**yes**	**15**	**2**

Every x that passes the test is printed: `1  3  7  15`.

The loop stops on the first no

```
int x = 1;
int count = 0;
while (x < 20) {
    printf("%d ", x);
    if (x % 3 == 0) {
        count++;
    }
    x = x * 2 + 1;
}
printf("\nx = %d, count = %d\n", x, count);
```

x	x < 20?	prints	count
1	yes	1	0
3	yes	3	1
7	yes	7	1
15	yes	15	2
31	**no**	**nothing**	**2**

x is now 31. `31 < 20` is false, so 31 is **never printed**. But x keeps it.

So the two lines are

```
int x = 1;
int count = 0;
while (x < 20) {
    printf("%d ", x);
    if (x % 3 == 0) {
        count++;
    }
    x = x * 2 + 1;
}
printf("\nx = %d, count = %d\n", x, count);
```

Output

```
1 3 7 15
x = 31, count = 2
```

x keeps the value that **failed**.

Write the whole line, words too.

Line 2 cost marks on more papers than any other line of the quiz.

Mistake 2: the counting program

```
int n, value;
int count = 0;
printf("How many numbers? ");
scanf("%d", &n);
for (int i = 1; i <= n; i++) {
    printf("Number: ");
    scanf("%d", &value);
    if (value % 2 == 0) {
        count++;
    }
}
printf("Even numbers: %d\n", count);
```

The screen

```
How many numbers? 5
Number: 4
Number: 7
Number: 10
Number: 3
Number: 8
Even numbers: 3
```

Four places, every time

Before	The loop	Inside	After
Start the counter. `int count = 0;`	Runs exactly `n` times. `i <= n`	Read, test, `count++;`	Print the result **once**.

One } in the wrong place

```
for (int i = 1; i <= n; i++) {
    printf("Number: ");
    scanf("%d", &value);
    if (value % 2 == 0) {
        count++;
    }
    printf("Even numbers: %d\n", count);
}
```

This `printf` is **inside** the loop.

Move it below the } that closes the loop.

C follows the braces, not the indent.

Run it

```
for (int i = 1; i <= n; i++) {
    printf("Number: ");
    scanf("%d", &value);
    if (value % 2 == 0) {
        count++;
    }
    printf("Even numbers: %d\n", count);
}
```

The screen

```
How many numbers? 3
Number: 4
Even numbers: 1
Number: 7
Even numbers: 1
Number: 10
Even numbers: 2
```

It prints after **every** number. The question wanted one line, at the end.

Mistake 3: the compiler finds the small things

```
int n, value;
int count = 0;
scanf("%d", &n);
for (i = 1; i =< n; i++) {
    scanf("%d"; &value);
    if (value % 2 == 0) {
        count++
    }
}
```

gcc -Wall says

```
error: 'i' undeclared

error: expected expression
before '<' token

error: expected ')'
before ';' token

error: expected ';'
before '}' token
```

`int i`, `<=`, a comma before &, a `;` after `count++`. Fix the **first** error, then compile again.

All three come back today

Every array loop today runs exactly N times.

Trace `i` and the box, row by row.

Count before, inside and after the loop.

At home: write Question 2 again, run it, and compare with your paper.

What Friday gave us

1. A variable lives inside its `{ }`: its **scope**
2. A function gets a **copy** of each value
3. A new value goes back with `return`
4. Recursion: a **base case** and a **step closer**

Recap: what is on the screen?

PREDICT

`add_ten` adds 10 to `n`.

What does `main` print?

```
void add_ten(int n) {
    n = n + 10;
}

int main(void) {
    int x = 5;
    add_ten(x);
    printf("%d\n", x);
    return 0;
}
```

Recap: what is on the screen?

PREDICT

`add_ten` adds 10 to `n`.

What does `main` print?

```
void add_ten(int n) {
    n = n + 10;
}

int main(void) {
    int x = 5;
    add_ten(x);
    printf("%d\n", x);
    return 0;
}
```

ANSWER

5

n was a **copy** of x.

Today's roadmap

1. What an array is: **many boxes, one name**
2. A loop over every box: `for (int i = 0; i < N; i++)`
3. Read, add up, print backwards
4. The mistakes C does **not** stop
5. Your turn: three small tasks

1 What an array is

Many values under one name

Five scores, five variables

```c
int main(void) {
    int s0 = 7, s1 = 9, s2 = 6, s3 = 10, s4 = 8;

    printf("%d %d %d %d %d\n", s0, s1, s2, s3, s4);
    int sum = s0 + s1 + s2 + s3 + s4;
    printf("average: %.1f\n", sum / 5.0);
    return 0;
}
```

Output

```
7 9 6 10 8
average: 8.0
```

Five names: `s0` to `s4`.

Every new score: a new name, and every line changes.

Now 35 students

Thirty-five variables? `s0`, `s1`, … `s34`?

Thirty-five names in the `printf`.

Thirty-five names in the sum.

And no loop can walk through them: they are 35 **different** names.

Five scores, one array

```c
int main(void) {
    int scores[5] = {7, 9, 6, 10, 8};

    int sum = 0;
    for (int i = 0; i < 5; i++) {
        printf("%d ", scores[i]);
        sum += scores[i];
    }
    printf("\n");
    printf("average: %.1f\n", sum / 5.0);
    return 0;
}
```

Output

```
7 9 6 10 8
average: 8.0
```

One name: `scores`.

Room for 5 `int` values.

A loop walks through it

```c
int main(void) {
    int scores[5] = {7, 9, 6, 10, 8};

    int sum = 0;
    for (int i = 0; i < 5; i++) {
        printf("%d ", scores[i]);
        sum += scores[i];
    }
    printf("\n");
    printf("average: %.1f\n", sum / 5.0);
    return 0;
}
```

`scores[i]`: the value in box `i`.

35 students: change the 5. The loop stays.

An array is a row of boxes

> **All the boxes have the same type and one name.**

Each box has a number: its **index**.

You reach a box with the name and the index: `scores[2]`.

> Like the lockers in a corridor: one row, one name on the row, a number on each door.

int scores[5] = {7, 9, 6, 10, 8};

index	0	1	2	3	4
`scores[ ]`	7	9	6	10	8

Five boxes. The index goes from **0** to **4**.

Read the boxes

PREDICT

```
int scores[5] = {7, 9, 6, 10, 8};
```

What is `scores[3]`?

What is `scores[0] + scores[4]`?

Read the boxes

PREDICT

```
int scores[5] = {7, 9, 6, 10, 8};
```

What is `scores[3]`?

What is `scores[0] + scores[4]`?

ANSWER

`scores[3]` is **10**: the fourth box.

`scores[0] + scores[4]` is 7 + 8 = **15**.

The first index is 0

An array of 5 has the indexes 0, 1, 2, 3, 4.

The first box: `scores[0]`.

The last box: `scores[4]`, **not** `scores[5]`.

An array of N: indexes 0 to N-1.

Make an array

```c
int main(void) {
    int a[5];

    a[0] = 10;
    a[1] = 20;
    a[4] = 50;

    printf("first: %d\n", a[0]);
    printf("second: %d\n", a[1]);
    printf("last: %d\n", a[4]);
    return 0;
}
```

`int`: the type of every box.

`a`: the name.

`[5]`: how many boxes.

Put a value into one box

```c
int main(void) {
    int a[5];

    a[0] = 10;
    a[1] = 20;
    a[4] = 50;

    printf("first: %d\n", a[0]);
    printf("second: %d\n", a[1]);
    printf("last: %d\n", a[4]);
    return 0;
}
```

`a[0] = 10;` works like `x = 10;`

`a[2]` and `a[3]` got nothing yet.

Use one box

```c
int main(void) {
    int a[5];

    a[0] = 10;
    a[1] = 20;
    a[4] = 50;

    printf("first: %d\n", a[0]);
    printf("second: %d\n", a[1]);
    printf("last: %d\n", a[4]);
    return 0;
}
```

Output

```
first: 10
second: 20
last: 50
```

`a[4]` is a value, like any `int` variable.

One box = one variable

Store	Read	Change
`a[1] = 20;`	`printf("%d", a[1]);`	`a[1] = a[1] + 5;` `a[1]++;`

> Everything you did with x you can do with `a[1]`.

Which index is February?

PREDICT

January is the first month.

Which index holds February?

Which holds December?

```
int days[12] = {31, 28, 31, 30,
                31, 30, 31, 31,
                30, 31, 30, 31};

printf("Feb: %d\n", days[1]);
printf("Dec: %d\n", days[11]);
```

Which index is February?

```
int days[12] = {31, 28, 31, 30,
                31, 30, 31, 31,
                30, 31, 30, 31};

printf("Feb: %d\n", days[1]);
printf("Dec: %d\n", days[11]);
```

Output

```
Feb: 28
Dec: 31
```

February: month 2, index **1**.

December: month 12, index **11**.

Month number = index + 1. The list in { } fills the boxes in order.

A short list

PREDICT

a has 5 boxes but only 2 values in the list.

What are the two lines?

```
int a[5] = {1, 2};
int b[5] = {0};

for (int i = 0; i < 5; i++) {
    printf("%d ", a[i]);
}
printf("\n");
for (int i = 0; i < 5; i++) {
    printf("%d ", b[i]);
}
printf("\n");
```

A short list

```
int a[5] = {1, 2};
int b[5] = {0};

for (int i = 0; i < 5; i++) {
    printf("%d ", a[i]);
}
printf("\n");
for (int i = 0; i < 5; i++) {
    printf("%d ", b[i]);
}
printf("\n");
```

Output

```
1 2 0 0 0
0 0 0 0 0
```

Missing values become **0**.

`= {0}`: every box 0.

Any type works

```
double prices[3] = {2.50, 1.20, 4.00};

prices[1] = 1.35;

for (int i = 0; i < 3; i++) {
    printf("item %d: %.2f AZN\n", i, prices[i]);
}
```

Output

```
item 0: 2.50 AZN
item 1: 1.35 AZN
item 2: 4.00 AZN
```

Three boxes of type `double`.

Making an array

1. `int a[5];` five boxes, no values yet
2. `int a[5] = {1, 2, 3, 4, 5};` five boxes, five values
3. `int a[5] = {0};` five boxes, all 0
4. The size is fixed: it cannot grow later
5. Index 0 to N-1

2 A loop over every box

```
for (int i = 0; i < N; i++)
```

The index can be a variable

`a[i]`: the box whose number is in `i`.

`i` is 0: `a[0]`. `i` is 3: `a[3]`.

A `for` loop makes `i` go 0, 1, 2, … N-1.

So one line inside the loop reaches **every** box.

Print every box

```
int a[5] = {12, 7, 30, 5, 18};

for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

Output

```
a[0] = 12
a[1] = 7
a[2] = 30
a[3] = 5
a[4] = 18
```

`i < 5`: stops after 4.

Step by step

```c
int a[5] = {12, 7, 30, 5, 18};

for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

i	a[i]	prints
0	12	a[0] = 12

`i` is 0, so `a[i]` is `a[0]`: 12.

Step by step

```
int a[5] = {12, 7, 30, 5, 18};

for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

i	a[i]	prints
0	12	a[0] = 12
1	7	a[1] = 7
2	**30**	**a[2] = 30**

Same line, new `i`, new box.

Step by step

```c
int a[5] = {12, 7, 30, 5, 18};

for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

i	a[i]	prints
0	12	a[0] = 12
1	7	a[1] = 7
2	30	a[2] = 30
3	5	a[3] = 5
4	18	a[4] = 18
5		**stop: 5 < 5 is false**

`i` reaches 5 and the loop stops. Five boxes, five lines, no box missed.

The array loop

```
for (int i = 0; i < 5; i++)
```

Starts at **0**: the first box.

Stops **before** 5: the last box is 4.

Learn it as one line. You will write it a hundred times.

Fill with a loop

```c
int main(void) {
    int sq[10];

    for (int i = 0; i < 10; i++) {
        sq[i] = i * i;
    }

    for (int i = 0; i < 10; i++) {
        printf("%d ", sq[i]);
    }
    printf("\n");
    return 0;
}
```

Box `i` gets `i * i`.

0, 1, 4, 9, …

Then print with a loop

```c
int main(void) {
    int sq[10];

    for (int i = 0; i < 10; i++) {
        sq[i] = i * i;
    }

    for (int i = 0; i < 10; i++) {
        printf("%d ", sq[i]);
    }
    printf("\n");
    return 0;
}
```

Output

```
0 1 4 9 16 25 36 49 64 81
```

Two loops: one fills, one prints.

Fill it with 1 to 10

PREDICT

Change one line so `sq` holds 1, 2, 3, …, 10.

What goes after `sq[i] =`?

```
for (int i = 0; i < 10; i++) {
    sq[i] = i * i;
}
```

Fill it with 1 to 10

PREDICT

Change one line so `sq` holds 1, 2, 3, ..., 10.

What goes after `sq[i] =`?

```
for (int i = 0; i < 10; i++) {
    sq[i] = i * i;
}
```

ANSWER

`sq[i] = i + 1;`

`i` goes 0 to 9, so `i + 1` goes 1 to 10.

3

Read, add up, print backwards

scanf into a box, a total, a loop going down

Read into the array

```c
int main(void) {
    int a[5];

    printf("Enter 5 numbers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < 5; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }
    return 0;
}
```

`&a[i]`: put the number **into** box `i`.

The & is needed, like &x.

Then print it back

```c
int main(void) {
    int a[5];

    printf("Enter 5 numbers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < 5; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }
    return 0;
}
```

Output

```
Enter 5 numbers: 4 8 15 16 23
a[0] = 4
a[1] = 8
a[2] = 15
a[3] = 16
a[4] = 23
```

Type them with spaces, or one per line.

Add them up

```c
int scores[5];

printf("Enter 5 scores: ");
for (int i = 0; i < 5; i++) {
    scanf("%d", &scores[i]);
}

int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
printf("sum: %d\n", sum);
printf("average: %.1f\n", sum / 5.0);
return 0;
```

Output

```
Enter 5 scores: 7 9 6 10 8
sum: 40
average: 8.0
```

> The total starts at 0, **before** the loop.

The running total

```c
int scores[5] = {7, 9, 6, 10, 8};

int sum = 0;
for (int i = 0; i < 5; i++) {
    printf("%d ", scores[i]);
    sum += scores[i];
}
```

i	scores[i]	sum
start		0
0	**7**	**7**

Class 6's running total, with `scores[i]` in place of `i`.

The running total

```
int scores[5] = {7, 9, 6, 10, 8};

int sum = 0;
for (int i = 0; i < 5; i++) {
    printf("%d ", scores[i]);
    sum += scores[i];
}
```

i	scores[i]	sum
start		0
0	7	7
1	9	16
2	6	22
3	10	32
4	**8**	**40**

After the loop `sum` is 40. The average: `40 / 5.0` = 8.0.

Print it backwards

PREDICT

The input is `1 2 3 4 5`.

Where must the second loop start?

Where must it stop?

```
int a[5];

printf("Enter 5 numbers: ");
for (int i = 0; i < 5; i++) {
    scanf("%d", &a[i]);
}

printf("Backwards:");
for (int i = 4; i >= 0; i--) {
    printf(" %d", a[i]);
}
printf("\n");
return 0;
```

Print it backwards

```c
int a[5];

printf("Enter 5 numbers: ");
for (int i = 0; i < 5; i++) {
    scanf("%d", &a[i]);
}

printf("Backwards:");
for (int i = 4; i >= 0; i--) {
    printf(" %d", a[i]);
}
printf("\n");
return 0;
```

Output

```
Enter 5 numbers: 1 2 3 4 5
Backwards: 5 4 3 2 1
```

Start at the **last** index: 4.

Go down, `i--`, while `i >= 0`.

Going down, step by step

```
printf("Backwards:");
for (int i = 4; i >= 0; i--) {
    printf(" %d", a[i]);
}
```

i	i >= 0?	prints
4	true	5
3	**true**	**4**

`i` starts at 4, the **last** index, and goes down by one.

Going down, step by step

```
printf("Backwards:");
for (int i = 4; i >= 0; i--) {
    printf(" %d", a[i]);
}
```

i	i >= 0?	prints
4	true	5
3	true	4
2	true	3
1	true	2
0	true	1
-1	**false**	**stop**

`i` reaches -1 and the loop stops. Box 0 was printed: >= and not >.

The size in one place

> **The number 5 is in three places. Change one, forget another: a bug.**

`#define N 5` at the top, after `#include`.

Everywhere below, gcc reads `N` as 5.

No `=` and no `;` on that line.

#define N 5

```c
#define N 5
int main(void) {
    int scores[N];
    printf("Enter %d scores: ", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &scores[i]);
    }
    int sum = 0;
    for (int i = 0; i < N; i++) {
        sum += scores[i];
    }
    printf("average: %.1f\n", (double) sum / N);
    return 0;
}
```

Output

```
Enter 5 scores: 7 9 6 10 8
average: 8.0
```

Change N once: the whole program follows.

Why keep every value?

QUESTION

Count the scores that are **above the average**.

Can you do it without an array?

Above the average

```
int sum = 0;
for (int i = 0; i < N; i++) {
    sum += scores[i];
}
double avg = (double) sum / N;

int above = 0;
for (int i = 0; i < N; i++) {
    if (scores[i] > avg) {
        above++;
    }
}
printf("average: %.1f\n", avg);
printf("above average: %d\n", above);
```

Output

```
Enter 5 scores: 7 9 6 10 8
average: 8.0
above average: 2
```

The average is known only **after** the last score.

Then we need every score **again**: the array kept them.

4 The mistakes C does not stop

Out of bounds, and b = a

Spot the bug

CAREFUL

Five values.

How many lines does it print?

```
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i <= 5; i++) {
        printf("%d\n", a[i]);
    }
    return 0;
}
```

One box too far

```c
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i <= 5; i++) {
        printf("%d\n", a[i]);
    }
    return 0;
}
```

Output

```
10
20
30
40
50
88
```

`i <= 5` also reads `a[5]`: there is none.

gcc -Wall says nothing

```c
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};

    printf("last: %d\n", a[5]);
    return 0;
}
```

Output

```
last: 136
```

It compiles with no warning.

It runs and prints a number.

Run it again: a different number.

gcc -O2 catches this one

```c
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};

    printf("last: %d\n", a[5]);
    return 0;
}
```

gcc -Wall -O2 says

```
warning: array
subscript 5 is above
array bounds of
'int[5]'
```

> Only with -O2, and not always. Never count on gcc to find this bug.

C does not check the index

> **Outside 0 to N-1 you read or write memory that is not yours.**

Reading: you get some number, a different one each run.

Writing: you may change **another** variable, or crash.

Your job: every loop `i = 0` and `i < N`.

Copy an array?

CAREFUL

`b = a;` should copy the three values.

Does it compile?

```
int a[3] = {1, 2, 3};
int b[3];

b = a;
printf("%d %d %d\n", b[0], b[1], b[2]);
```

b = a does not compile

```
int a[3] = {1, 2, 3};
int b[3];

b = a;
printf("%d %d %d\n", b[0], b[1], b[2]);
```

gcc says

```
error: assignment
to expression with
array type
```

A whole array cannot be assigned.

Copy box by box

```c
int a[3] = {1, 2, 3};
int b[3];

for (int i = 0; i < 3; i++) {
    b[i] = a[i];
}
b[0] = 100;

printf("a: %d %d %d\n", a[0], a[1], a[2]);
printf("b: %d %d %d\n", b[0], b[1], b[2]);
```

Output

```
a: 1 2 3
b: 100 2 3
```

A loop copies each box.

b is a real copy: changing it leaves a alone.

Three mistakes

mistake	what happens	fix
`i <= 5` for 5 boxes	reads `a[5]`: a wrong number	`i < 5`
`a[5]` as the last box	a wrong number, no error	`a[4]`
`b = a;`	does not compile	a loop: `b[i] = a[i];`

5 Your turn

Three small programs

Task 1: a week of temperatures

YOUR TURN

Read the temperatures of the **7** days into a `double` array.

Print them in one line.

Print their average.

Input: `18 21 19 23 25 22 20`

Task 1: one answer

```c
double t[DAYS];

for (int i = 0; i < DAYS; i++) {
    printf("Day %d: ", i + 1);
    scanf("%lf", &t[i]);
}

double sum = 0;
for (int i = 0; i < DAYS; i++) {
    printf("%.1f ", t[i]);
    sum += t[i];
}
printf("\n");
printf("average: %.1f\n", sum / DAYS);
```

Output

```
Day 1: 18
Day 2: 21
Day 3: 19
Day 4: 23
Day 5: 25
Day 6: 22
Day 7: 20
18.0 21.0 19.0 23.0 25.0 22.0 20.0
average: 21.1
```

Task 2: double every value

YOUR TURN

Start from

```
int a[6] = {3, 8, 1, 5, 9, 2};
```

Double every value **in** the array, then print it.

Task 2: one answer

```c
int main(void) {
    int a[6] = {3, 8, 1, 5, 9, 2};

    for (int i = 0; i < 6; i++) {
        a[i] = a[i] * 2;
    }

    for (int i = 0; i < 6; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

Output

```
6 16 2 10 18 4
```

```c
a[i] = a[i] * 2;
```
or `a[i] *= 2;`

Task 3: how many are even?

YOUR TURN

Start from

```
int a[8] = {4, 7, 10, 3, 6, 6, 1, 12};
```

Count the even values.

Task 3: one answer

```c
int main(void) {
    int a[8] = {4, 7, 10, 3, 6, 6, 1, 12};

    int even = 0;
    for (int i = 0; i < 8; i++) {
        if (a[i] % 2 == 0) {
            even++;
        }
    }
    printf("even values: %d\n", even);
    return 0;
}
```

Output

```
even values: 5
```

A counter inside an `if`: class 6.

Today in one slide

1. `int a[5];` five boxes, one name, one type
2. Index from **0** to **N-1**: `a[0]` to `a[4]`
3. The loop: `for (int i = 0; i < N; i++)`
4. `scanf("%d", &a[i]);` reads into a box
5. C does not check the index. `b = a;` does not compile

Before Friday

1. **Knowledge check 9** on Blackboard: today's slides
2. **Project 1** (Blackjack): due **Sun 11 October, 23:59**
3. **Quiz 3: Fri 16 October**, classes 7 to 10
4. Today's code: Blackboard, Week 5 module, **Class 9 code**

> Next class: search an array, find the smallest and the largest, sort it.